1a) Explain the 3 OOP principles

The Three OOP Principles

Abstraction

- Data Abstraction is the property by virtue of which only the **essential details are displayed to the user.** The trivial or **non-essential units are not displayed to the user**.

Ex: A car is viewed as a car rather than its individual components.

Abstraction in Java is the process in which we only show essential details/functionality to the user. The non-essential implementation details are not displayed to the user.

2: Encapsulation

- It is defined as the **wrapping up of data under a single unit**. It is the mechanism that binds together the code and the data it manipulates.
- Another way to think about **encapsulation is that it is a protective shield that prevents the data from being accessed by the code outside this shield.**
- Technically, in encapsulation, **the variables or the data in a class is hidden from any other class and can be accessed only through any member function of the class in which they are declared.**
- In encapsulation, the data in a class is hidden from other classes, which is similar to what **data-hiding** does. **So, the terms "encapsulation" and "data-hiding" are used interchangeably.**
- **Encapsulation can be achieved by declaring all the variables in a class as private and writing public methods in the class to set and get the values of the variables.**

Therefore, any other code that is not a member of the class cannot access a private method or variable.

Since the private members of a class may only be accessed by other parts of your program through the class's public methods

3: Inheritance

- Inheritance is an important pillar of OOP (Object Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class. **We are achieving inheritance by using extends keyword**. Inheritance is also known as "**is-a**" relationship.
- Let us discuss some frequently used important terminologies:
- **Superclass:** The class whose features are inherited is known as superclass (**also known as base or parent class**).
- **Subclass:** The **class that inherits the other class is known as subclass** (also known as derived or extended or child class). **The subclass can add its own fields and methods in addition to the superclass fields and methods.**
- **Reusability:** Inheritance supports the concept of "reusability", i.e. **when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class.** By doing this, we are reusing the fields and methods of the existing class.

1b) Write a program to find the biggest among 3 numbers using ternary operator (syntax also)

```
// Java program to find largest
// among two numbers using
// ternary operator
class GFG
{
public static void main(String args[])
{
    // variable declaration
    int n1 = 5, n2 = 10, max;

    // Largest among n1 and n2
    max = (n1 > n2) ? n1 : n2;

    // Print the largest number
    System.out.println("Largest number between " + n1 +
                " and " + n2 + " is " + max + ". " );
}
}
```

2a) Explain the arithmetic operator with an example

- **Arithmetic Operators**

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators:

Operator	Result
+	Addition
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
–=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

```
/*Simple example program demonstrates the arithmetic operators.
  It also illustrates the difference between floating-point division and integer division
*/
class BasicMath {
public static void main(String args[])
{
// arithmetic using integers
System.out.println("Integer Arithmetic");
    int a = 1 + 1;
    int b = a * 3;
    int c = b / 4;
    int d = c - a;
    int e = -d;
    System.out.println("a = " + a);
    System.out.println("b = " + b);
    System.out.println("c = " + c);
    System.out.println("d = " + d);
    System.out.println("e = " + e);
    // arithmetic using doubles
    System.out.println("\nFloating Point Arithmetic");
    double da = 1 + 1;
    double db = da * 3;
    double dc = db / 4;
    double dd = dc - a;
    double de = -dd;
    System.out.println("da = " + da);
    System.out.println("db = " + db);
    System.out.println("dc = " + dc);
    System.out.println("dd = " + dd);
    System.out.println("de = " + de);
}
```

```
F:\Java\First-year>java BasicMath
Integer Arithmetic
a = 2
b = 6
c = 1
d = -1
e = 1

Floating Point Arithmetic
da = 2.0
db = 6.0
dc = 1.5
dd = -0.5
de = 0.5
```

2b) Explain the automatic type promotion in expressions with example

Automatic Type Promotion in Java

When different types of values are used in an expression, Java automatically **converts smaller data types to larger types** to avoid errors. This is called **automatic type promotion**.

Rules:

- `byte`, `short`, and `char` → promoted to `int`
- If one value is `long` → result is `long`
- If one value is `float` → result is `float`
- If one value is `double` → result is `double`

Example:

Java Copy code

```java
byte a = 10;
int b = 5;
int c = a + b;  // 'a' is promoted to
int
System.out.println(c);  // Output: 15
```

3a) Explain the switch statement with example

A `switch` statement in Java allows you to execute **one block of code** from many options based on the value of a variable. It's often used when checking a variable against several constant values (like numbers or strings).

Example;

```
import java.util.Scanner;

public class SwitchExample {
public static void main(String[] args)
                {
          Scanner scanner = new
        Scanner(System.in);

      System.out.print("Enter a number
            (1-7): ");
        int day = scanner.nextInt();

              switch (day) {
                    case 1:

     System.out.println("Monday");
                      break;
                  case 2:

     System.out.println("Tuesday");
                      break;
                  case 3:

    System.out.println("Wednesday");
                      break;
                  case 4:

     System.out.println("Thursday");
                      break;
                  case 5:

      System.out.println("Friday");
                      break;
                  case 6:

     System.out.println("Saturday");
                      break;
                  case 7:

      System.out.println("Sunday");
                      break;
                 default:

System.out.println("Invalid input! Please
   enter a number between 1 and 7.");
                     }

               scanner.close();
                   }
                }
```

3b) Explain while and do while loop with example

In Java, both `while` and `do-while` loops are used for repeated execution of code **as long as a condition is true**, but they behave slightly differently.

---

1. while loop

- **Condition is checked first.**
- **If false at the start, the loop body won't execute at all.**

Java Copy code

```
public class WhileExample {
    public static void main(String[]
args) {
        int i = 1;

        while (i <= 5) {
            System.out.println("Count: "
+ i);
            i++;
        }
    }
}
```

Output:

Copy code

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

---

2. do-while loop

- **Loop body is executed first**, then the condition is checked.
- **It always runs at least once**, even if the condition is false.

Java

Copy code

```
public class DoWhileExample {
    public static void main(String[]
args) {
        int i = 1;

        do {
            System.out.println("Count: "
+ i);
            i++;
        } while (i <= 5);
    }
}
```

Output (same as above):

Copy code

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```